Tổng hợp và biên soạn: .Lê Thành Nam.

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

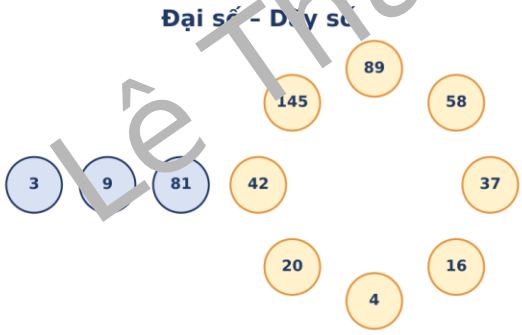
TOÁN HỌC TRONG TIN HỌC

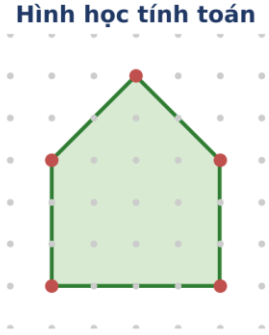
Số học · Tổ hợp - Xác suất · Đại số · Hình học tính toán

TẬP 1 · LÝ THUYẾT









Lê Thành Nam

Chương 1. SỐ HỌC

Chương 2. TỔ HỢP - XÁC SUẤT

Chương 3. ĐẠI SỐ - DÃY SỐ

Chương 4. HÌNH HỌC TÍNH TOÁN

LỜI NÓI ĐẦU

Mở một đề thi học sinh giỏi Tin học cấp tỉnh bất kì, em sẽ thấy toán học có mặt ở khắp nơi: đếm ước, tìm số nguyên tố, tính số dư của một lũy thừa khổng lồ, đếm số cách chọn, tính diện tích một mảnh đất có toạ độ cho trước. Những bài này thường không đòi hỏi cấu trúc dữ liệu phức tạp, nhưng lại là nơi nhiều học sinh mất điểm nhất: chương trình đúng với ví dụ nhưng chạy quá thời gian khi n lớn, hoặc cho kết quả sai vì tràn số mà không hề báo lỗi.

Nguyên nhân là toán trong Tin học không dừng ở chỗ "biết công thức". Em cần biết **vì sao** công thức đúng, **đếm được** chương trình thực hiện bao nhiêu phép tính, và **cài đặt** sao cho mọi giá trị trung gian đều nằm gọn trong kiểu dữ liệu. Bộ tài liệu này được viết để rèn đủ ba kĩ năng đó, từ những bài đếm ước đơn giản nhất đến các bài phân loại của đề thi cấp tỉnh. Bộ sách gồm ba tập đi cùng nhau:

• **Tập 1: Lý thuyết** (cuốn sách này) gồm 12 bài học chia thành 4 chương: Số học; Tổ hợp - Xác suất; Đại số - Dãy số; Hình học tính toán.

• **Tập 2: Bài tập** gồm 36 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật.

• **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm của từng cách làm.

Mỗi bài học không đưa ra công thức ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi ví dụ đều được chạy tay; mọi chương trình trong sách đều đã được biên dịch, chạy thử và đối chiếu với lời giải vét cạn trên nhiều bộ dữ liệu.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Tên biến được viết đầy đủ để em đọc code như đọc một đoạn văn.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thanh Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối và phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm (hoặc sai), và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: quan sát, công thức, chạy tay, chứng minh, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu để kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
CÔNG THỨC	Công thức, định lí hoặc thuật toán cốt lõi.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là O(n), O(√n), O(log n), ...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một bảng chạy tay, hãy che cột kết quả lại và tự tính trước. Mỗi khi gặp một công thức, hãy tự trả lời ba câu hỏi: công thức nói gì, vì sao nó đúng, và các giá trị trung gian lớn tới đâu (có tràn kiểu dữ liệu không). Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Bảng tra nhanh: ràng buộc gợi ý độ phức tạp (với giới hạn khoảng 10^8 phép tính mỗi giây):

Ràng buộc	Độ phức tạp cần đạt	Ví dụ trong sách
$n \le 10$	$O(n!)$	Sinh hoán vị (Bài 6)
$n \le 20$	$O(2^n \cdot n)$	Duyệt tập con (Bài 6, Bài 7)

Lê Thành Nam

Ràng buộc	Độ phức tạp cần đạt	Ví dụ trong sách
$n \le 5\,000$	$O(n^2)$	Thử mọi đoạn, mọi cặp
$n \le 10^6$	$O(n)$ hoặc $O(n \log n)$	Sàng, mảng cộng dồn (Bài 2, Bài 8)
$n \le 10^{12}$	$O(\sqrt{n})$	Đếm ước, phân tích thừa số (Bài 1)
$n \le 10^{18}$	$O(\log n)$ hoặc $O(1)$	Lũy thừa nhanh, công thức (Bài 4, Bài 9, Bài 10)

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. SỐ HỌC **6**

Bài 1. Ước số và số nguyên tố 7
Bài 2. Sàng Eratosthenes và phân tích thừa số nguyên tố 16
Bài 3. Ước chung lớn nhất, bội chung nhỏ nhất và thuật toán Euclid 24
Bài 4. Đồng dư, lũy thừa nhanh và xử lý tràn số 30

CHƯƠNG 2. TỔ HỢP - XÁC SUẤT **38**

Bài 5. Quy tắc đếm, tổ hợp và tam giác Pascal 39
Bài 6. Sinh cấu hình và quay lui 48
Bài 7. Nguyên lí Dirichlet, bao hàm - loại trừ và xác suất 57

CHƯƠNG 3. ĐẠI SỐ - DÃY SỐ **65**

Bài 8. Công thức tổng, cấp số và mảng cộng dồn 66
Bài 9. Dãy truy hồi và chu kỳ 73
Bài 10. Hệ cơ số và thao tác bit 79

CHƯƠNG 4. HÌNH HỌC TÍNH TOÁN **85**

Bài 11. Điểm, vector và tích có hướng 86
Bài 12. Đa giác: diện tích, định lí Pick và điểm trong đa giác 93

Lê Thanh Nam

CHƯƠNG 1. SỐ HỌC

Số học là "sân nhà" của đề thi học sinh giỏi cấp tỉnh: gần như năm nào cũng có ít nhất một câu về ước, bội, số nguyên tố hay số dư. Những bài này nhìn thì quen thuộc, vì em đã học ở môn Toán từ lớp 6, nhưng đưa vào máy tính thì nảy sinh hai câu hỏi mới: làm sao kiểm tra một số cỡ 10^{12} trong chớp mắt, và làm sao tính số dư của một số có hàng triệu chữ số khi kiểu long long chỉ chứa được 19 chữ số. Chương này trả lời hai câu hỏi đó bằng bốn công cụ: ghép cặp ước để chỉ duyệt tới $\sqrt{n}$, sàng Eratosthenes, thuật toán Euclid, và lũy thừa nhanh theo modulo.

TRONG CHƯƠNG NÀY

Bài 1. Ước số và số nguyên tố: Vì sao chỉ cần duyệt tới $\sqrt{n}$?

Bài 2. Sàng Eratosthenes và phân tích thừa số nguyên tố

Bài 3. Ước chung lớn nhất, bội chung nhỏ nhất và thuật toán Euclid

Bài 4. Đồng dư, lũy thừa nhanh và xử lý tràn số

Lê Thành Nam

BÀI 1 Ước số và số nguyên tố

Học xong bài này, em sẽ:

- ✓ Hiểu vì sao các ước của n luôn đi thành **cặp** (d, n/d), và nhờ đó chỉ cần duyệt d tới √n.
- ✓ Đếm ước, liệt kê ước theo thứ tự tăng dần, kiểm tra số nguyên tố trong O(√n).
- ✓ Biết xử lý đúng trường hợp n là **số chính phương** và trường hợp n = 1.
- ✓ Biết viết điều kiện vòng lặp `d * d <= n` thay cho `d <= sqrt(n)` để tránh sai số thực.

KHỞI ĐỘNG

Cô giáo có 36 chiếc ghế và muốn xếp chúng thành một hình chữ nhật kín (mỗi hàng có số ghế như nhau). Có thể xếp 1 hàng 36 ghế, 2 hàng 18 ghế, 3 hàng 12 ghế, ... Hỏi có bao nhiêu cách xếp, nếu coi "a hàng, b ghế mỗi hàng" và "b hàng, a ghế mỗi hàng" là khác nhau?

Em sẽ nhận ra đây chính là câu hỏi "36 có bao nhiêu ước?". Với 36 thì liệt kê tay rất nhanh. Nhưng nếu số ghế là 963 761 198 400 thì sao? Cuối bài này, chương trình của em sẽ trả lời câu hỏi đó (6 720 cách) trong chưa đầy một phần trăm giây.

BÀI TOÁN. ĐẾM ƯỚC [DEMUOC]

Cho số nguyên dương n. Hãy đếm số ước nguyên dương của n và cho biết n có phải là số nguyên tố hay không.

Dữ liệu vào:

- Một dòng duy nhất chứa số nguyên n ($1 \le n \le 10^{12}$).

Kết quả:

- Dòng thứ nhất: số ước của n.
- Dòng thứ hai: CO nếu n là số nguyên tố, KHONG nếu ngược lại.

Dữ liệu vào	Kết quả	Giải thích
`12`	`6` `KHONG`	Các ước: 1, 2, 3, 4, 6, 12.
`13`	`2` `CO`	13 chỉ có hai ước là 1 và 13.
`1`	`1` `KHONG`	Số 1 có đúng một ước, không phải số nguyên tố.

Nhắc lại: số nguyên tố là số tự nhiên lớn hơn 1 chỉ có hai ước là 1 và chính nó. Vậy "n là số nguyên tố" tương đương với "n có **đúng hai** ước". Hai yêu cầu của đề bài thực chất là một.

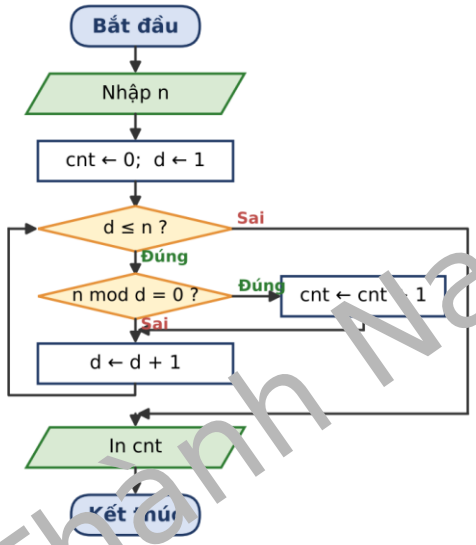
Lê Thành Nam

1. Ý tưởng ngây thơ: Thử mọi số từ 1 đến n

Ước của n là các số d mà n chia hết cho d, tức `n % d == 0`. Mọi ước đều nằm trong đoạn [1, n], nên cách tự nhiên nhất là thử lần lượt d = 1, 2, ..., n và đếm.

Code ý chính: Đếm ước ngây thơ

```
long long cnt = 0;
for (long long d = 1; d <= n; d++)
    if (n % d == 0) cnt++;
```



Hình 1.1. Sơ đồ khối của cách đếm ước ngây thơ

Chương trình chạy đúng với cả ba ví dụ. Nhưng hãy thử với n = 10^{12}.

VÌ SAO CÁCH NGÂY THƠ CHẬM? ĐẾM TỪNG BƯỚC

• Vòng lặp chạy đúng n lần, mỗi lần một phép chia lấy dư: độ phức tạp **O(n)**.

• Một máy chấm thông thường thực hiện khoảng 10^8 phép tính đơn giản mỗi giây. Với n = 10^6: khoảng 0,01 giây. Với n = 10^9: khoảng 10 giây. Với n = 10^{12}: khoảng 10^4 giây, gần **ba giờ đồng hồ**.

• Phép chia lấy dư trên số 64 bit còn chậm hơn phép cộng nhiều lần, nên con số thực tế còn tệ hơn.

BẾ TẮC

Đề bài cho n tới 10^{12} và giới hạn 1 giây. Thử từng số từ 1 đến n là không thể. Nhưng hãy nhìn lại danh sách ước của 12: 1, 2, 3, 4, 6, 12. Có thật sự phải thử tới tận 12 mới tìm được ước 12 không, khi mà ngay từ lúc thử d = 1 ta đã biết 12 / 1 = 12 cũng là một ước?

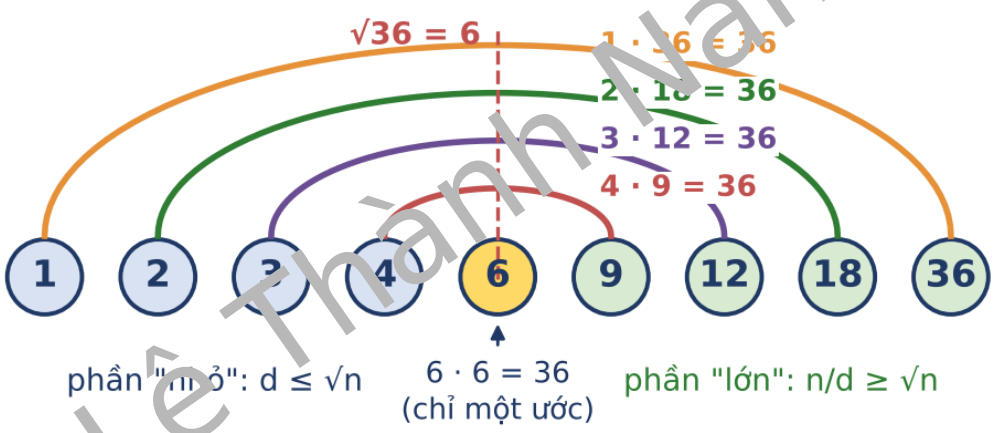
Lê Thành Nam

2. Phá bế tắc: Ước đi thành cặp

Bước 1 Viết các ước thành từng cặp

Mỗi khi d là ước của n, thì e = n / d cũng là một ước của n, vì d · e = n. Ta gọi (d, e) là một **cặp ước**. Hãy viết các ước của 36 thành từng cặp có tích bằng 36:

d	e = 36 / d	d · e	Nhận xét
1	36	36	d < e
2	18	36	d < e
3	12	36	d < e
4	9	36	d < e
6	**6**	36	**d = e = √36**
9	4	36	lặp lại cặp (4, 9)



Hình 1.2. Các ước của 36 ghép thành cặp có tích bằng 36. Mọi cặp đều có một số ≤ 6 = √36

Từ d = 9 trở đi, các cặp chỉ lặp lại những cặp đã gặp. Ranh giới nằm đúng ở √36 = 6.

Bước 2 Chứng minh: luôn có một số không vượt quá √n

VÌ SAO CHỈ CẦN DUYỆT TỚI √n?

Giả sử d · e = n và cả hai số đều lớn hơn √n. Khi đó d · e > √n · √n = n, mâu thuẫn. Vậy trong mỗi cặp ước (d, e), **số nhỏ hơn luôn không vượt quá √n**.

Do đó, nếu ta duyệt d từ 1 đến √n và mỗi khi gặp ước d thì ghi nhận luôn cả cặp (d, n/d), ta sẽ gặp đủ mọi cặp, và mỗi cặp đúng một lần.

Lê Thành Nam

Bước 3 Cẩn thận với số chính phương

Trong bảng trên, cặp (6, 6) chỉ gồm **một** ước chứ không phải hai. Điều này xảy ra khi và chỉ khi n là số chính phương và d = √n. Nếu ta cứ cộng 2 cho mỗi cặp, số 36 sẽ bị đếm thành 10 ước thay vì 9.

> **CÔNG THỨC ĐẾM ƯỚC BẰNG GHÉP CẶP**
>
> Duyệt d = 1, 2, ... trong khi d · d ≤ n. Nếu n chia hết cho d:
>
> - nếu d · d = n: cộng **1** (cặp (√n, √n) chỉ có một ước);
> - ngược lại: cộng **2** (hai ước khác nhau d và n/d).
>
> Số lần lặp: khoảng √n. Với n = 10^{12}, chỉ có 10^{6} lần lặp.

Bước 4 Chạy tay

Với n = 36, vòng lặp dừng khi d · d > 36, tức sau d = 6:

d	d · d ≤ 36?	36 mod d	Cộng thêm	cnt
1	1 ≤ 36	0	2 (ước 1 và 36)	**2**
2	4 ≤ 36	0	2 (ước 2 và 18)	**4**
3	9 ≤ 36	0	2 (ước 3 và 12)	**6**
4	16 ≤ 36	0	2 (ước 4 và 9)	**8**
5	25 ≤ 36	1	0	**8**
6	36 ≤ 36	0	1 (vì 6 · 6 = 36)	**9**
7	49 > 36	dừng		

Kết quả 9 ước, đúng với danh sách 1, 2, 3, 4, 6, 9, 12, 18, 36.

Code ý chính: Đếm ước trong O(√n)

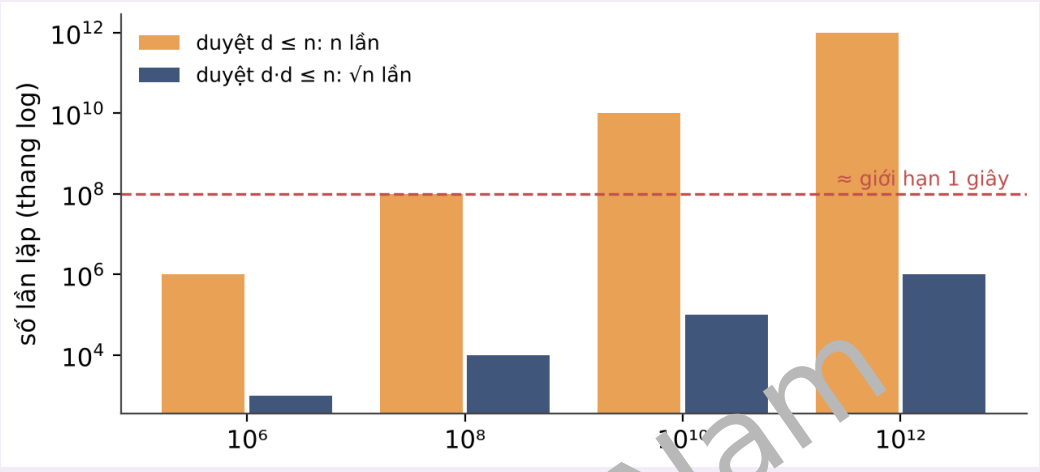
```
long long cnt = 0;
for (long long d = 1; d * d <= n; d++) {
    if (n % d != 0) continue;
    if (d * d == n) cnt += 1;
    else cnt += 2;
}
```

Lê Thành Nam

CÁCH MỚI NHANH TỚI ĐÂU?

• Vòng lặp chạy khoảng √n lần: độ phức tạp **O(√n)**.

• n = 10^{12}: 10^6 lần lặp, chạy trong vài mili giây. So với cách ngây thơ (10^{12} lần lặp), ta nhanh hơn **một triệu lần**.



Hình 1.3. *Số lần lặp của hai cách (thang logarit). Đường đỏ là giới hạn khoảng 10^8 phép tính trong 1 giây*

3. Kiểm tra số nguyên tố

Khi chỉ cần biết n có nguyên tố hay không, ta còn làm được nhanh hơn nữa: **dừng ngay** khi tìm thấy một ước d với 2 ≤ d ≤ √n.

VÌ SAO CHỈ CẦN THỬ ƯỚC TỪ 2 ĐẾN √n?

Nếu n > 1 là hợp số thì n = d · e với 1 < d ≤ e < n. Theo lập luận ở mục 2, d ≤ √n. Vậy **mọi hợp số n đều có một ước d thỏa 2 ≤ d ≤ √n**. Ngược lại, nếu không tìm thấy ước nào trong đoạn đó thì n là số nguyên tố.

Code ý chính: Kiểm tra nguyên tố trong O(√n)

```
bool isPrime(long long n) {
    if (n < 2) return false;            // 0 va 1 khong phai so nguyen to
    for (long long d = 2; d * d <= n; d++)
        if (n % d == 0) return false;   // tim thay uoc: dung ngay
    return true;
}
```

Với hợp số, hàm thường dừng rất sớm (một nửa số tự nhiên chia hết cho 2, một phần ba chia hết cho 3, ...). Chỉ khi n thật sự là số nguyên tố, vòng lặp mới phải chạy hết √n lần. Đây là trường hợp xấu nhất mà đề thi thường cố ý đưa vào, ví dụ n = 999 999 999 989 (số nguyên tố lớn nhất nhỏ hơn 10^{12}).

Lê Thành Nam

Cải tiến nhỏ: chỉ thử số lẻ. Sau khi kiểm tra riêng d = 2, ta chỉ cần thử các số lẻ 3, 5, 7, ... vì nếu n lẻ thì n không có ước chẵn. Số lần lặp giảm một nửa, nhưng độ phức tạp vẫn là O(√n). Ở cấp tỉnh, cải tiến này hiếm khi là ranh giới giữa được và mất điểm; điều quan trọng là phải duyệt tới √n chứ không phải tới n.

4. Liệt kê ước theo thứ tự tăng dần: Bài toán LIETKEUOC

Một biến thể hay gặp: in ra **tất cả** các ước của n theo thứ tự tăng dần. Khi duyệt d tăng dần từ 1 tới √n, các ước "nhỏ" d xuất hiện theo thứ tự **tăng**, còn các ước "lớn" n/d xuất hiện theo thứ tự **giảm**. Ta lưu hai danh sách riêng rồi in danh sách nhỏ, sau đó in danh sách lớn theo chiều ngược lại.

Bước	Danh sách nhỏ (tăng)	Danh sách lớn (giảm)
d = 1	1	36
d = 2	1, 2	36, 18
d = 3	1, 2, 3	36, 18, 12
d = 4	1, 2, 3, 4	36, 18, 12, 9
d = 6	1, 2, 3, 4, 6	(6 = 36/6: không thêm lần nữa)

In ra: 1 2 3 4 6, rồi đảo ngược danh sách lớn: 9 12 18 36. Không cần sắp xếp, độ phức tạp vẫn là O(√n).

5. Chương trình hoàn chỉnh

Phần đọc tệp được viết theo kiểu "có tệp thì đọc tệp": khi chấm bằng Themis với tệp DEMUOC.INP, chương trình đọc và ghi tệp; khi nộp lên trang chấm trực tuyến (đọc từ bàn phím), điều kiện fopen sai nên chương trình đọc chuẩn như bình thường.

Lê Thành Nam

DEMUOC.CPP: Đếm ước và kiểm tra nguyên tố, O(√n)

```cpp
#include <cstdio>
#include <iostream>
using namespace std;

int main() {
    if (fopen("DEMUOC.INP", "r")) {
        freopen("DEMUOC.INP", "r", stdin);
        freopen("DEMUOC.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    long long cnt = 0;
    for (long long d = 1; d * d <= n; d++) {
        if (n % d != 0) continue;
        long long e = n / d;          // d va e = n / d la mot cap uoc
        if (d == e) cnt += 1;         // n la so chinh phuong, d = e = can n
        else cnt += 2;
    }
    cout << cnt << '\n';
    cout << (cnt == 2 ? "CO" : "KHONG") << '\n';
    return 0;
}
```

Vài chi tiết đáng chú ý:

- Biến n và d đều kiểu **long long**. Với n tới 10^{12}, kiểu int (tối đa khoảng $2{,}1 \cdot 10^9$) không chứa nổi n; và ngay cả d chỉ tới 10^6 thì tích `d * d` cũng phải tính trong long long.
- Điều kiện `d * d <= n` dùng hoàn toàn số nguyên. Nếu viết `d <= sqrt(n)`, hàm sqrt trả về số thực và có thể sai lệch một chút ở các số chính phương lớn, làm vòng lặp dừng sớm hoặc muộn một bước.
- Sau khi đếm xong, "n nguyên tố" chính là "cnt = 2". Số 1 có cnt = 1 nên tự động được in KHONG.

LIETKEUOC.CPP: Liệt kê ước tăng dần, O(√n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    if (fopen("LIETKEUOC.INP", "r")) {
        freopen("LIETKEUOC.INP", "r", stdin);
        freopen("LIETKEUOC.OUT", "w", stdout);
    }
    long long n;
    cin >> n;
    vector<long long> small, large;
    for (long long d = 1; d * d <= n; d++) {
        if (n % d != 0) continue;
        small.push_back(d);                    // d tang dan
        if (d != n / d) large.push_back(n / d);  // n / d giam dan
    }
    for (size_t i = 0; i < small.size(); i++) cout << small[i] << ' ';
    for (int i = (int)large.size() - 1; i >= 0; i--)
        cout << large[i] << ' ';
    cout << '\n';
    return 0;
}
```

Lê Thành Nam

6. Mở rộng

Mở rộng 1: Tổng các ước. Thay vì đếm, ta cộng: gặp cặp (d, n/d) thì cộng d + n/d, riêng d · d = n thì chỉ cộng d. Tổng các ước thực sự (không kể n) dùng để xét số hoàn hảo (6 = 1 + 2 + 3), số dư, số thiếu. Đây là bài 1.2 ở Tập 2.

Mở rộng 2: Đếm ước của nhiều số. Nếu đề bài cho q = 10^6 số, mỗi số tới 10^7, thì O(q · √a) ≈ 3 · 10^9 phép tính là quá chậm. Ta cần chuẩn bị trước bằng sàng (Bài 2).

Mở rộng 3: Số có đúng 3 ước. Một số có số ước lẻ khi và chỉ khi nó là số chính phương (vì các ước ghép cặp, chỉ có cặp (√n, √n) là "lẻ loi"). Từ đó suy ra: n có đúng 3 ước khi và chỉ khi n = p^2 với p nguyên tố. Câu hỏi "đếm các số có đúng 3 ước trong [L, R]" với R tới 10^{12} trở thành "đếm số nguyên tố trong [√L, √R]", chỉ còn khoảng 10^6 số cần xét. Đây là bài 1.3 ở Tập 2.

Mở rộng 4: Phân tích thừa số nguyên tố trong O(√n). Cũng bằng lập luận "có ước ≤ √n", ta chia n lần lượt cho d = 2, 3, 4, ... chừng nào còn chia hết, dừng khi d · d > n; nếu phần còn lại lớn hơn 1 thì đó là một thừa số nguyên tố. Bài 2 sẽ trình bày kĩ và dùng nó để có công thức số ước.

BẪY THƯỜNG GẶP

- Duyệt `d <= n` thay vì `d * d <= n`: đúng nhưng chậm, chỉ ăn subtask nhỏ.
- Cộng 2 cho mọi cặp, quên trường hợp d · d = n: số chính phương bị đếm thừa một ước.
- Dùng kiểu int cho n hoặc cho tích `d * d`: tràn số khi n > 2,1 · 10^9, vòng lặp chạy sai mà không báo lỗi.
- Viết `d <= sqrt(n)` với n lớn: sai số thực có thể làm thiếu ước √n của số chính phương.
- Coi 1 là số nguyên tố, hoặc quên xét riêng n < 2 trong hàm isPrime.

GHI NHỚ

- Ước luôn đi thành cặp (d, n/d), và trong mỗi cặp có một số ≤ √n.
- Đếm ước, liệt kê ước, kiểm tra nguyên tố đều làm được trong **O(√n)**.
- Số chính phương có cặp (√n, √n) chỉ gồm một ước: số ước của nó là số lẻ.
- Viết điều kiện lặp bằng số nguyên: `d * d <= n`, dùng long long.

LUYỆN TẬP

Bài	Tên	Gợi ý liên hệ
1.1	XẾP GHẾ	Đếm số cách xếp n ghế thành hình chữ nhật a × b (a ≤ b) và tìm chu vi nhỏ nhất. Mỗi cặp ước là một cách.

Lê Thanh Nam

Bài	Tên	Gợi ý liên hệ
1.2	SỐ HOÀN HẢO	Tính tổng các ước thực sự bằng ghép cặp. Coi chừng số chính phương và n = 1.
1.3	BA ƯỚC	Đếm số có đúng 3 ước trong [L, R], R ≤ 10^{12}. Số đó có dạng gì?

Tự kiểm tra nhanh

1. Với n = 10^{12}, vòng lặp `for (d = 1; d * d <= n; d++)` chạy bao nhiêu lần?

(Đáp số: 10^6 lần.)

2. Số 100 có bao nhiêu ước? Liệt kê theo cặp.

(Đáp số: 9 ước; các cặp (1, 100), (2, 50), (4, 25), (5, 20), (10, 10).)

3. Có bao nhiêu số tự nhiên không vượt quá 100 có đúng ba ước?

(Đáp số: 4 số, là 4, 9, 25, 49.)

Lê Thành Nam